AUTONOMOUS SOFTWARE EXPERIMENTS ON HERA: ANNEX A - HERA INTERFACE API DOCUMENTATION

Reference	ESA-HERA-TECS-IF-2026-002046
Issue/Revision	1.0
Date of Issue	19/06/2026

Table of Contents

1. PAYLOAD CONTROL

These functions control the acquisition and storage of images from the Payload subsystems.

1.1. Hera_AFC_AcquireSingleImage

```
error_code_t Hera_AFC_AcquireSingleImage(uint32 exp_us);
```

Initiates a single image acquisition using the AFC. The resulting image is stored in a temporary internal buffer and can be accessed via `Hera_AFC_GetImageBuffer()`.

1.1.1. Parameters

- `exp_us`: The desired exposure time specified in microseconds.

1.1.2. Timing & Blocking

- Blocking: Yes.
- Duration: 7 seconds + `exp_us`. The function returns only after the acquisition is complete.

1.1.3. Return Values

- `AFC_IMAGE_REQUEST_COMPLETED`: The acquisition was successful and the image is ready in the buffer.
- `AFC_ERROR`: An acquisition failure occurred. Note: The Hera platform will automatically stop the experiment software if this error occurs.

1.2. Hera_AFC_StoreImage

```
error_code_t Hera_AFC_StoreImage(void);
```

Transfers the most recently acquired AFC image via `Hera_AFC_AcquireSingleImage()` from the temporary internal buffer to the Spacecraft Mass Memory Unit. Only images stored using this function will be scheduled for downlink to Ground.

Important: If this function is not called, the image data in the temporary buffer will be overwritten by the next call to `Hera_AFC_AcquireSingleImage()`.

1.2.1. Timing & Blocking

- Blocking: Yes.
- Duration: 10 seconds.

1.2.2. Return Values

- `AFC_IMAGE_COPY_COMPLETED`: The image was successfully transferred to Mass Memory.
- `AFC_ERROR`: A transfer failure occurred. Note: The Hera platform will automatically stop the experiment software if this error occurs.

1.3. Hera_AFC_GetImageBuffer

```
uint8* Hera_AFC_GetImageBuffer(void);
```

Retrieves the memory address of the temporary buffer containing the latest image acquired by the AFC.

1.3.1. Timing & Blocking

- Blocking: No (returns immediately).

1.3.2. Return Values

- A pointer (`uint8*`) to the start of the raw image data.

1.4. Hera_HS_AcquireImage

```
error_code_t Hera_HS_AcquireImage(uint32 exp_ms);
```

Triggers an acquisition sequence for the HyperScout hyperspectral camera. The image data is automatically transferred to the Spacecraft Mass Memory Unit for downlink. No separate store command is required.

1.4.1. Parameters

- `exp_ms`: The desired exposure time specified in milliseconds.

1.4.2. Timing & Blocking

- Blocking: Yes.
- Duration: 15 minutes.

1.4.3. Return Values

- `HS_ACQUIRE_COPY_COMPLETED`: The image was successfully acquired and transferred to Mass Memory.
- `HS_ERROR`: A failure occurred. Note: The Hera platform will automatically stop the experiment software if this error occurs.

1.5. Hera_TIRI_AcquireImage

```
error_code_t Hera_TIRI_AcquireImage(void);
```

Triggers an acquisition sequence for the TIRI thermal camera. The image data is automatically transferred to the Spacecraft Mass Memory Unit for downlink. No separate store command is required.

1.5.1. Timing & Blocking

- Blocking: Yes.
- Duration: 10 minutes.

1.5.2. Return Values

- `TIRI_ACQUIRE_COPY_COMPLETED`: The image was successfully acquired and transferred to Mass Memory.
- `TIRI_ERROR`: A failure occurred. Note: The Hera platform will automatically stop the experiment software if this error occurs.

2. SYSTEM CONTROL

2.1. Hera_Sleep

```
error_code_t Hera_Sleep(uint32 seconds);
```

Suspends the execution of Core 1 for a specified duration. This is used to idle the processor between operations.

2.1.1. Parameters

- `seconds`: The duration to sleep in seconds.

2.1.2. Timing & Blocking

- Blocking: Yes. The function returns only after the timer expires.

3. TELEMETRY & REPORTING

3.1. Hera_HK_Report

```
error_code_t Hera_HK_Report(uint16 sid, void* pData, uint16 size);
```

Generates a Housekeeping (HK) packet to be stored in the Mass Memory Unit. The structure of the packet is defined by the Structure ID (SID).

Refer to the PUS-C Standard and distributed examples for packet formatting.

Specific SID ranges are allocated to each experiment.

3.1.1. Parameters

- `sid`: The Structure ID defining the housekeeping packet layout.
- `pData`: Pointer to the data payload.
- `size`: Size of the payload in bytes (Max: 256 bytes including headers).

3.1.2. Timing & Blocking

- Blocking: Yes.
- Duration: 1 second.

3.2. Hera_Event_Report

```
error_code_t Hera_Event_Report(uint16 event_id, void* pData, uint16 size);
```

Generates an asynchronous Event packet to be stored in the Mass Memory Unit.

Refer to the PUS-C Standard and distributed examples for event formatting.

Specific Event ID ranges are allocated to each experiment.

3.2.1. Parameters

- `event_id`: The unique identifier for the event.
- `pData`: Pointer to the event data payload.
- `size`: Size of the payload in bytes (Max: 50 bytes including headers).

3.2.2. Timing & Blocking

- Blocking: Yes.
- Duration: 1 second.

3.3. Hera_Science_Report

```
error_code_t Hera_Science_Report(uint16 apid, uint8 type,
                                 uint8 subtype, void* pData,
```

```
uint16 size);
```

Generates a generic Science Data packet to be stored in the Mass Memory Unit.

APID, Type, and Subtype ranges are allocated to each experiment.

3.3.1. Parameters

- `apid`: Application Process ID.
- `type`: PUS Service Type.
- `subtype`: PUS Service Subtype.
- `pData`: Pointer to the science data payload.
- `size`: Size of the payload in bytes (Max: 2048 bytes including headers).

3.3.2. Timing & Blocking

- Blocking: Yes.
- Duration: 1 second.

4. DATA POOL ACCESS

These functions retrieve system parameters from the Spacecraft Data Pool.

4.1. Available Functions

```
uint8   Hera_Read_Parameter_uint8(uint16 parameter_ref);
uint16  Hera_Read_Parameter_uint16(uint16 parameter_ref);
uint32  Hera_Read_Parameter_uint32(uint16 parameter_ref);
int8    Hera_Read_Parameter_int8(uint16 parameter_ref);
int16   Hera_Read_Parameter_int16(uint16 parameter_ref);
int32   Hera_Read_Parameter_int32(uint16 parameter_ref);
float32 Hera_Read_Parameter_float32(uint16 parameter_ref);
float64 Hera_Read_Parameter_float64(uint16 parameter_ref);
```

Reads the current value of a specific parameter from the spacecraft. The developer must select the function that corresponds to the data type of the target parameter.

4.2. Parameters

- `parameter_ref`: The unique identifier for the parameter (See Annex B for a description of the data pool parameters).

4.3. Return Values

- Returns the value of the requested parameter (cast to the appropriate return type) or an error code if the read operation fails.